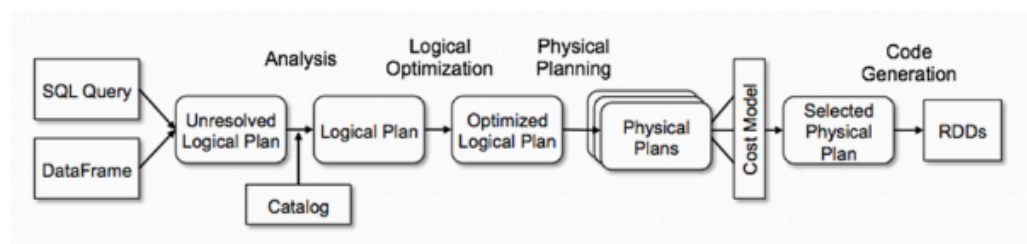


17 Catalyst Optimizer in Apache Spark (SQL Engine)

Catalyst Optimizer



Apache Spark's Catalyst Optimizer is a powerful component that enhances the performance of Spark applications by optimizing the execution of data queries. It uses a set of pre-configured rules or custom-defined rules in the optimization layer to improve query performance.

1 What is Catalyst Optimizer?

Catalyst Optimizer is a query optimization framework introduced in Apache Spark. It's designed to optimize the execution of data queries by applying a series of transformations to the query plan. These transformations are based on a set of rules, which can be pre-configured or custom-defined.

📌 The Catalyst Optimizer takes a query, expressed in Spark's DataFrame or SQL API, and transforms it into an optimized physical plan for execution.

📌 It uses information about the data and the query itself (like filters or joins) to apply optimizations.

📌 The goal of these optimizations is to reduce the amount of data that needs to be processed and to simplify the operations themselves.

2 How Does Catalyst Optimizer Work?

The Catalyst Optimizer works in several stages:

- 📌 Analysis: In this stage, the Catalyst Optimizer analyzes the logical plan to resolve references to named expressions, like resolving column names or table names.
- 📌 Logical Optimization : The optimizer applies rule-based optimizations to the logical plan, such as predicate pushdown or constant folding.
- 📌 Physical Planning : The optimizer generates several physical plans from the logical plan and chooses the most efficient one based on cost estimation.
- 📌 Code Generation : Finally, the optimizer generates executable bytecode to run the query.

3 Key Points of Catalyst Optimizer

- ✅ The Catalyst Optimizer is a key component of Apache Spark that significantly improves the performance of data queries.
- ✅ It works by applying a series of rule-based transformations to the query plan, reducing the amount of data that needs to be processed and simplifying the operations.
- ✅ The Catalyst Optimizer supports both pre-configured rules and custom-defined rules, providing flexibility for different optimization strategies.

```
# analysis exception
df_orders.select("oort").show()

# optimized logical plan
df_orders.filter(col("order_status") == 'PLACED').select("order_id","state").filter(col("state") == 'Goa').explain(True)

df_orders.filter(col("order_status") == 'PLACED').select("order_id","state").filter(col("state") == 'Goa').explain()

df_orders.select("*").filter(col("order_status") == 'PLACED').select("order_id","state").filter(col("state") == 'Goa').explain(True)
```

It works with SQL and dataframe API and not with low level rdd api.

What is Tungsten Optimizer

Tungsten is a **performance optimization engine in Spark SQL** that focuses on **CPU and memory efficiency**, not query logic (that is Catalyst's job).

Think of it as

Catalyst decides *what* to do

Tungsten decides *how fast* it runs

Main goals

- Reduce JVM overhead
- Improve CPU efficiency
- Use memory more efficiently

Key features

1. Memory management

- Avoids Java objects and GC pressure
- Uses **binary format** instead of JVM objects
- Data is stored in compact memory layout

Why it matters

Less garbage collection → more stable performance

2. Whole Stage Code Generation

- Generates **Java bytecode** for entire query stages
- Fuses multiple operators (filter, project, join) into **one loop**
- Avoids virtual function calls

Result

Fewer function calls → better CPU cache usage

In Spark UI you will see

`WholeStageCodegen` in physical plan

3. Cache friendly data structures

- Uses columnar and binary formats
- Improves CPU cache locality
- Faster iteration over data

4. Vectorized processing

- Processes data in batches instead of row by row
- Especially useful for scans and aggregations

Where Tungsten is used

- Spark SQL
 - DataFrame and Dataset APIs
- (Not RDD API)

Tungsten vs Catalyst

Aspect	Catalyst	Tungsten
Type	Logical optimizer	Execution engine
Focus	Query plan optimization	Runtime performance
Works on	Logical and physical plans	Memory, CPU, bytecode